EBSD ODF Analyzer

Microstructure and crystallographic texture from EBSD scans

User Manual

Version 1.0

Multi-format input	.ang (TSL text), .osc (EDAX binary), .ctf / h5ebsd
Microstructure	IQ / CI / IPF maps, configurable boundaries, ASTM E2627 grain size
Texture	MTEX-style ODF (kernel or harmonic), phi2 sections, alpha / gamma fibers
Multiphase	per-phase symmetry, per-phase ODF, phase boundaries
Export	PowerPoint report with selectable content

1 Overview

EBSD ODF Analyzer turns an indexed EBSD scan into a complete microstructure and texture analysis, then exports the result as a PowerPoint report. It is a desktop application: you move through five steps in order, adjusting parameters in the left sidebar and inspecting results on the right.

The five steps

1 Load & Resample	Read the scan file and resample the hexagonal measurement grid onto a square pixel grid.
2 Microstructure	Orientations, misorientations, IQ / CI / IPF maps, grain-boundary classes and grain segmentation.
3 Grain Size	Grain-size distributions and the ASTM E2627 grain-size number G.
4 Texture (ODF)	Orientation distribution function, phi2 sections and alpha / gamma fiber profiles.
5 Report	Export the selected results to a PowerPoint file.

Supported input formats

.ang	TSL / EDAX text export. Euler angles in radians (Bunge).
.osc	EDAX OIM binary. Read natively — no external converter needed.
.ctf, .h5, .oh5	Read through orix (HKL / h5ebsd family).

Crystal symmetry and the phase list are read from the file header automatically. If a file carries no symmetry, the app falls back to m-3m (cubic) and writes a warning to the log.

2 Installation and launch

Option A — standalone executable (no install)

Copy the EBSD_Analyzer folder anywhere and run EBSD_Analyzer.exe. Everything needed is bundled; Python does not have to be installed.

```
standalone_exe_new/
  EBSD_Analyzer/
    EBSD_Analyzer.exe      <- double-click this
    _internal/ ...         <- bundled libraries (do not delete)
```

Option B — reproducible environment with pixi

pixi builds the exact, locked dependency set from conda-forge only, so no Anaconda licence is required.

```
cd standalone_ebsd
pixi install        # create the locked environment
pixi run app        # launch the GUI
```

Verifying an installation

Both forms accept a headless self-test that runs the whole pipeline (both ODF engines, grain segmentation, a multiphase check and a PowerPoint export) and prints a pass or fail line.

```
EBSD_Analyzer.exe --selftest        # frozen build
pixi run python app.py --selftest   # pixi environment
```

3 Step 1 — Load & Resample

Choose the scan file, then press Run. The scan's hexagonal grid is resampled onto a square grid by nearest-neighbour lookup so that all later image operations work on regular pixels.

Settings

Scan file	Path to the scan. The Browse dialog lists every supported format by default (.ang, .osc, .ctf, .h5 / .oh5 / .hdf5 / .dream3d), with per-format filters available.
Grid ratio	Square pixel size as a multiple of the hex step. 1.0 keeps the native resolution; larger values downsample.
Sample-frame rotation	Optional rotation about ND, RD or TD applied before texture analysis.
Column indices (advanced)	Override the column order if a file deviates from the standard TSL layout.

Choosing what goes into the report

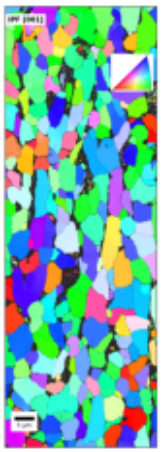
The right-hand panel lists every item that can be placed in the PowerPoint export. Ticked items are shown in bold. The selection and all option values can be stored as a preset:

Save preset…	Write the current selection and every parameter value to a JSON file.
Load preset…	Restore a saved setup. Presets deliberately do not store the scan path, so one preset can be reused across datasets.

Preset files are plain JSON and can be edited by hand — useful for sharing a standard analysis recipe across a group.

4 Step 2 — Microstructure

This step computes orientations, neighbour misorientations, the orientation and quality maps, grain-boundary classes, and the grain segmentation used later for grain size.



IPF [001] map — TSL colour key and OIM micron marker

Key settings

Crystal symmetry	'(from data)' reads the point group from the file header. Choose a value explicitly to override it.
Maps to show	Any combination of IQ, CI, IPF [100] / [010] / [001], a custom IPF direction, boundary map, IPF+boundaries, grain map and phase map. Defaults: IQ, CI and the three IPF maps.
CI map style	Greyscale (default) or a colour map.
Overlays	Independently toggle the intensity legend, the scale bar and the axis labels. Turning the legend on or off never resizes or rescales the micrograph — the figure grows instead, so the image itself is pixel-identical either way.
CI threshold	Points with CI below this value are treated as unindexed and dropped from every downstream calculation.
Grain-splitting angle	Misorientation above which neighbouring pixels belong to different grains. Default 15°. This is set on its own — it is NOT derived from the boundary classes below.
Clean low-CI pixels	Off by default. When on, low-CI pixels adopt their best-indexed neighbour's orientation (grain dilation).

How maps are annotated

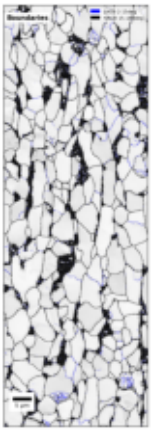
Micrographs follow the OIM presentation convention. Painted on the image: the map name, the TSL stereographic-triangle colour key on IPF maps, and a micron marker — a solid black bar on a white box with the length and unit printed beneath it. The intensity legend for scalar maps (IQ, CI) is drawn in a strip BELOW the image, never over the data.

Grain-boundary display classes

Boundaries are not fixed to a single low/high pair. You choose how many classes to define, and each class has its own angular range, colour and line thickness. These control DISPLAY only.

Count	Number of boundary classes (1 to 6).
name	Label used in map legends, e.g. LAGB or HAGB.
from lo° / to hi°	A boundary is drawn where lo° <= misorientation < hi°. The range is closed at the low end and open at the high end, so adjacent classes never double-count a segment.
colour / width	Line colour and thickness for that class on every map that draws boundaries.

Adding or editing a display class never changes the grain count. Grains are split by the separate 'Grain-splitting angle' setting (default 15°). Defaults here are LAGB 2–15° and HAGB 15–180°.



Boundary map — each configured class drawn in its own colour and thickness

5 Multiphase scans

When the file header declares more than one phase, every stage becomes phase-aware, following the same conventions as MTEX:

- Each phase keeps its own crystal symmetry; IPF colouring uses the symmetry of the phase that each pixel belongs to.
- Misorientation is only computed between neighbours of the same phase. A boundary between two different phases is treated as a grain boundary, so grains never span a phase change.
- A separate ODF is computed for each phase, each with its own texture index J, phi2 sections and fiber profiles.
- A phase map is added to the results automatically.

Two phases can share a point group and still be different phases — FCC and BCC are both cubic m-3m and are distinguished by their lattice parameters, which the reader also extracts.

Phase map of a two-phase scan (ferrite + silver)

6 Step 3 — Grain Size

Grains come from a union-find segmentation: neighbouring valid pixels are merged while their misorientation stays below the grain-splitting angle set in Step 2 (default 15 deg). Grains smaller than the minimum pixel count are discarded.

Histogram bins	Bin count for the distribution charts.
Min grain (px)	Grains below this area are ignored (Step 2 setting).
Exclude edge grains	Ignore grains touching the scan border, which are only partly measured.

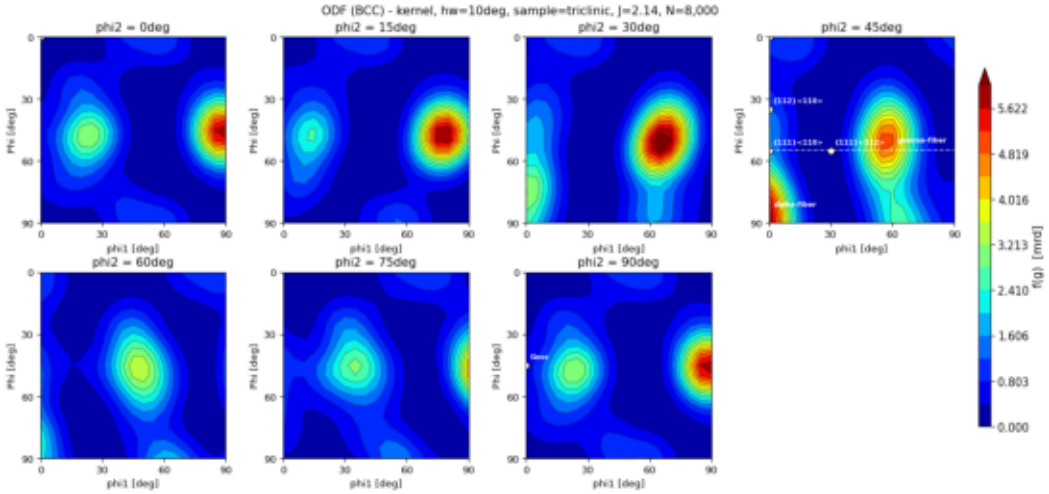
Reported quantities include the number-average and area-weighted equivalent-circle diameters, the mean grain area, the cumulative area-weighted distribution, and the ASTM E2627 grain-size number G.

7 Step 4 — Texture (ODF)

The orientation distribution function is estimated with a de la Vallée Poussin kernel, the same approach MTEX uses. Two mathematically equivalent engines are available and agree to within numerical noise.

kernel	Direct kernel summation over orientations. Needs only orix.
harmonic	Wigner-D / generalised spherical harmonic series — MTEX's internal algorithm. Uses the spherical and quaternionic packages.
Kernel halfwidth	Kernel width in degrees. 10° is the MTEX default; smaller values give sharper, noisier textures.
Specimen symmetry	triclinic (none), monoclinic, or orthorhombic — orthorhombic folds the ODF about RD, TD and ND.
N sample	Number of orientations sampled for speed. Increase for a smoother ODF at the cost of runtime.
phi2 sections	Which constant-phi2 slices to plot.

Results include the texture index J (J = 1 for a random texture), the phi2 section maps in multiples of a random distribution, and the alpha and gamma fiber intensity profiles.



ODF phi2 sections with ideal component markers and fiber traces

8 Step 5 — Report

Exports the analysis to a PowerPoint file. Content follows the selection made in Step 1.

Report contents	All results, microstructure only, or texture only.
Output file	Destination .pptx path.
Slide size / DPI	Slide geometry in inches and the raster resolution of embedded figures.

9 Troubleshooting

Wrong or odd IPF colours	Check the crystal symmetry in Step 2. If the header was missing, the app defaults to m-3m and logs a warning.
Too few or too many grains	Adjust the Grain-splitting angle in Step 2, the CI threshold and the minimum grain size. Adding boundary display classes does not affect the grain count.
Speckled maps	Raise the CI threshold, or enable 'Clean low-CI pixels' to fill unindexed points from their neighbours.
ODF looks noisy	Increase N sample or the kernel halfwidth.
Texture index J near 1	The material is nearly randomly oriented; this is a result, not an error.
Application will not start	Run the self-test from a terminal (--selftest) to see which component fails.

10 Reference — file layout

```
standalone_ebsd/
  app.py               GUI entry point
  worker.py            background pipeline thread
  pixi.toml            locked dependency definition
  ebsd_engine/
    ebsd_read.py       multi-format reader (.ang/.osc/orix)
    config.py          all settings (Config dataclass)
    microstructure.py  load, misorientation, grains, grain size
    odf.py             ODF engines (kernel / harmonic)
    plotting.py        figure builders
    report.py          PowerPoint builder
  ui/                  GUI widgets, steps and theme
  packaging/           build helpers and this manual generator
```

The .osc reader is derived from MTEX's BSD-licensed loadEBSD_osc.m and was validated against matching .ang exports: identical point counts and Euler / position / CI values to float32 precision.